

WellnessLog.in — Deployment Guide (Docker + Caddy)

Deploys WellnessLog.in on your Hostinger VPS using Docker Compose with Caddy reverse proxy (label-based routing).

Prerequisites

- **Hostinger VPS** at `157.173.222.202` with Docker + Caddy already running
 - **caddy_network** Docker network already created
 - **DNS A record:** `wellnesslog.in` → `157.173.222.202` (already done)
 - **MX records:** Still pointing to Hostinger mail servers (for `update@wellnesslog.in`)
-

1. Clone the Repo

```
ssh root@157.173.222.202

mkdir -p /opt/apps/wellnesslog
cd /opt/apps/wellnesslog

git clone -b hostinger https://github.com/vikram11/wellnesslogin.git .
```

The app code is inside `nextjs_space/`.

2. Configure Environment

```
cd nextjs_space
cp .env.production .env
nano .env
```

Fill in these values:

Variable	What to do
DB_PASSWORD	Set a strong random password (e.g., <code>openssl rand -base64 24</code>)
OLLAMA_API_KEY	Already pre-filled
OLLAMA_MODEL	Already pre-filled (<code>gemma4:31b-cloud</code>)
NEXT_PUBLIC_VAPID_PUBLIC_KEY	Generate: <code>npx web-push generate-vapid-keys</code> — copy public key
VAPID_PRIVATE_KEY	From same command — copy private key
SMTP_PASS	Password for <code>update@wellnesslog.in</code> (from Hostinger email)

3. Deploy

```
cd /opt/apps/wellnesslog/nextjs_space

# Build and start
docker compose up -d --build

# Watch logs during first build (takes a few minutes)
docker compose logs -f app
```

Once you see `Ready` on `http://0.0.0.0:3000` , the app is up.

4. Initialize Database

The database schema needs to be pushed on first deploy:

```
# Run prisma db push inside the running container
docker compose exec app yarn prisma db push

# (Optional) Seed with sample data
docker compose exec app yarn tsx scripts/seed.ts
```

5. Verify

```
# Check containers are running
docker ps

# Verify app is on caddy_network
docker network inspect caddy_network | grep wellnesslog

# Check Caddy picked up the labels
docker logs caddy-proxy --tail 50
```

Then open **<https://wellnesslog.in>** in a private/incognito window.

6. Push Notification Cron Jobs

The app has 3 daily push reminders. Set up cron on the VPS:

```
crontab -e
```

Add:

```
# Morning reminder - 8:00 AM Central
0 8 * * * docker exec nextjs_space-app-1 wget -q -O- --post-data='{ "title": "Good Morning! ☀️", "body": "Time to log your morning BP and meds." }' --header='Content-Type: application/json' http://localhost:3000/api/push/send

# Midday reminder - 1:00 PM Central
0 13 * * * docker exec nextjs_space-app-1 wget -q -O- --post-data='{ "title": "Midday Check-in 🕒", "body": "Have you taken your midday medications?" }' --header='Content-Type: application/json' http://localhost:3000/api/push/send

# Evening reminder - 8:00 PM Central
0 20 * * * docker exec nextjs_space-app-1 wget -q -O- --post-data='{ "title": "Evening Wrap-up 🌙", "body": "Log your evening BP and meds before bed." }' --header='Content-Type: application/json' http://localhost:3000/api/push/send
```

Set VPS timezone:

```
timedatectl set-timezone America/Chicago
```

Note: The container name `nextjs_space-app-1` may differ. Check with `docker ps`.

7. Architecture

```
Internet → Caddy (ports 80/443, auto-SSL)
          ↓ (caddy_network)
          app (Next.js :3000)
          ↓ (internal network)
          db (PostgreSQL :5432)
```

Service	Network	Exposed?
app	caddy_network + internal	Via Caddy only
db	internal only	No — internal to Docker

8. Ongoing Maintenance

Pull updates & rebuild

```
cd /opt/apps/wellnesslog
git pull origin hostinger
cd nextjs_space
docker compose up -d --build
```

View logs

```
docker compose logs -f app      # App logs
docker compose logs -f db      # Database logs
```

Database backup

```
# One-time backup
docker compose exec db pg_dump -U wellnesslog wellnesslog > backup_$(date +%Y%m%d).sql

# Automated daily backup via cron
0 3 * * * docker exec nextjs_space-db-1 pg_dump -U wellnesslog wellnesslog > /opt/
backups/wellnesslog_$(date +%Y%m%d).sql
```

Restart

```
docker compose restart app      # Just the app
docker compose restart          # Everything
```

9. What's Ready vs. What You Fill In

Component	Status	Action
✓ LLM (Ollama Cloud)	Pre-filled	Just verify API key in <code>.env</code>
✓ Docker Compose	Ready	<code>docker compose up -d --build</code>
✓ PostgreSQL	Auto-created	Schema push needed once (step 4)
✓ Caddy/SSL	Auto via labels	Nothing to do
✓ Email config	Pre-filled	Set <code>SMTP_PASS</code> in <code>.env</code>
⚙️ VAPID keys	Empty	Generate and paste into <code>.env</code>
⚙️ DB password	Placeholder	Set a strong password in <code>.env</code>
⚙️ Push cron	Instructions above	Add 3 crontab lines

That's it! 🎉